

Introduction to C Programming

2

Objectives

In this chapter, you'll:

- Write simple C programs.
- Use simple input and output statements.
- Use the fundamental data types.
- Learn computer memory concepts.
- Use arithmetic operators.
- Learn the precedence of arithmetic operators.
- Write simple decision-making statements.
- Begin focusing on secure C programming practices.

Avoid Single-Argument `printf`s³

One such guideline is to *avoid using `printf` with a single string argument*. If you need to display a string that *terminates with a newline*, use the **`puts` function**, which displays its string argument followed by a newline character. For example, in Fig. 2.1, line 8

```
printf( "Welcome to C!\n" );
```

should be written as:

```
puts( "Welcome to C!" );
```

We did not include `\n` in the preceding string because `puts` adds it automatically.

If you need to display a string *without* a terminating newline character, use `printf` with *two* arguments—a `"%s"` format control string and the string to display. The **`%s` conversion specifier** is for displaying a string. For example, in Fig. 2.3, line 8

```
printf( "Welcome " );
```

should be written as:

```
printf( "%s", "Welcome " );
```

Although the `printf`s in this chapter as written are actually *not* insecure, these changes are responsible coding practices that will eliminate certain security vulnerabilities as we get deeper into C—we'll explain the rationale later in the book. From this point forward, we use these practices in the chapter examples and you should use them in your exercise solutions.

`scanf` and `printf`, `scanf_s` and `printf_s`

We introduced `scanf` and `printf` in this chapter. We'll be saying more about these in subsequent Secure C Coding Guidelines sections, beginning in Section 3.13. We'll also discuss `scanf_s` and `printf_s`, which were introduced in C11.

-
3. For more information, see CERT C Secure Coding rule FIO30-C (www.securecoding.cert.org/confluence/display/seccode/FIO30-C.+Exclude+user+input+from+format+strings). In Chapter 6's Secure C Programming section, we'll explain the notion of user input as referred to by this CERT guideline.

Summary

Section 2.1 Introduction

- The C language facilitates a structured and disciplined approach to computer-program design.

Section 2.2 A Simple C Program: Printing a Line of Text

- **Comments** (p. 72) begin with `//`. Comments **document programs** (p. 72) and improve program readability. C also supports **multi-line comments** that begin with `/*` and end with `*/` (p. 73).
- Comments do not cause the computer to perform any action when the program is run. They're ignored by the C compiler and do not cause any machine-language object code to be generated.
- Lines beginning with `#` are processed by the **preprocessor** before the program is compiled. The **`#include` directive** tells the preprocessor (p. 73) to include the contents of another file.

- The **<stdio.h> header** (p. 73) contains information used by the compiler when compiling calls to standard input/output library functions such as `printf`.
- The function **main** is a part of every C program. The parentheses after `main` indicate that `main` is a program building block called a **function** (p. 73). C programs contain one or more functions, one of which must be `main`. Every program in C begins executing at the function `main`.
- Functions can return information. The keyword **int** to the left of `main` indicates that `main` “returns” an integer (whole-number) value.
- Functions can receive information when they’re called upon to execute. The **void** in parentheses after `main` indicates that `main` does not receive any information.
- A **left brace**, `{`, begins the **body** of every function (p. 73). A corresponding **right brace**, `}`, ends each function (p. 73). This pair of braces and the portion of the program between the braces is called a **block**.
- The **printf function** (p. 74) instructs the computer to display information on the screen.
- A **string** is sometimes called a **character string**, a **message** or a **literal** (p. 74).
- Every **statement** (p. 74) must end with a **semicolon** (also known as the **statement terminator**; p. 74).
- In `\n` (p. 74), the backslash (`\`) is called an **escape character** (p. 74). When encountering a backslash in a string, the compiler looks ahead at the next character and combines it with the backslash to form an **escape sequence** (p. 74). The escape sequence `\n` means **newline**.
- When a newline appears in the string output by a `printf`, the newline causes the cursor to position to the beginning of the next line on the screen.
- The **double backslash** (`\\`) **escape sequence** can be used to place a single backslash in a string.
- The escape sequence `\"` represents a literal double-quote character.

Section 2.3 Another Simple C Program: Adding Two Integers

- A **variable** (p. 77) is a location in memory where a value can be stored for use by a program.
- Variables of type **int** (p. 77) hold **integer values**, i.e., **whole numbers** such as 7, -11, 0, 31914.
- All variables must be defined with a name and a **data type** before they can be used in a program.
- A variable name in C is any valid **identifier**. An identifier (p. 77) is a series of characters consisting of letters, digits and underscores (`_`) that does not begin with a digit.
- C is **case sensitive** (p. 77)—uppercase and lowercase letters are different in C.
- Standard Library function **scanf** (p. 78) can be used to obtain input from the standard input, which is usually the keyboard.
- The **scanf format control string** (p. 78) indicates the type(s) of data that should be input.
- The **%d conversion specifier** (p. 78) indicates that the data should be an integer (the letter `d` stands for “decimal integer”). The `%` in this context is treated by `scanf` (and `printf`) as a special character that begins a conversion specifier.
- The arguments that follow `scanf`’s format control string begin with an **ampersand** (`&`)—called the **address operator** (p. 78)—followed by a variable name. The ampersand, when combined with a variable name, tells `scanf` the location in memory at which the variable is located. The computer then stores the value for the variable at that location.
- Most calculations are performed in **assignment statements** (p. 79).
- The `=` operator and the `+` operator are **binary operators**—each has two operands (p. 79).
- In a `printf` that specifies a format control string as its first argument the conversion specifiers indicate placeholders for data to output.

Section 2.4 Memory Concepts

- Variable names correspond to locations in the computer's memory. Every **variable** has a **name**, a **type** and a **value**.
- Whenever a value is placed in a memory location, the value replaces the previous value in that location; thus, placing a new value into a memory location is said to be **destructive** (p. 80).
- When a value is read from a memory location, the process is said to be **nondestructive** (p. 81).

Section 2.5 Arithmetic in C

- In algebra, if we want to multiply a times b , we can simply place these single-letter variable names side by side, as in ab . In C, however, if we were to do this, ab would be interpreted as a single, two-letter name (or identifier). Therefore, C (like other programming languages, in general) requires that multiplication be explicitly denoted by using the $*$ operator, as in $a * b$.
- **Arithmetic expressions** (p. 81) in C must be written in **straight-line form** (p. 82) to facilitate entering programs into the computer. Thus, expressions such as “ a divided by b ” must be written as a/b , so that all operators and operands appear in a straight line.
- **Parentheses** are used to group terms in C expressions in much the same manner as in algebraic expressions.
- C evaluates arithmetic expressions in a precise sequence determined by the following **rules of operator precedence** (p. 82), which are generally the same as those followed in algebra.
- **Multiplication, division and remainder** operations are applied first. If an expression contains several multiplication, division and remainder operations, evaluation proceeds from left to right. Multiplication, division and remainder are said to be on the same level of precedence.
- **Addition and subtraction** operations are evaluated next. If an expression contains several addition and subtraction operations, evaluation proceeds from left to right. Addition and subtraction also have the same level of precedence, which is lower than the precedence of the multiplication, division and remainder operators.
- The rules of operator precedence specify the order C uses to evaluate expressions. The **associativity** (p. 83) of the operators specifies whether they evaluate from left to right or from right to left.

Section 2.6 Decision Making: Equality and Relational Operators

- Executable C statements either perform **actions** or make **decisions**.
- C's **if statement** (p. 85) allows a program to make a decision based on the truth or falsity of a statement of fact called a condition (p. 85). If the condition is met (i.e., the condition is **true**; p. 85) the statement in the body of the **if** statement executes. If the condition isn't met (i.e., the condition is **false**; p. 85) the body statement does not execute. Whether the body statement is executed or not, after the **if** statement completes, execution proceeds with the next statement after the **if** statement.
- Conditions in **if** statements are formed by using the **equality** and **relational operators** (p. 85).
- The relational operators all have the same level of precedence and associate left to right. The equality operators have a lower level of precedence than the relational operators and they also associate left to right.
- To avoid confusing assignment ($=$) and equality ($==$), the assignment operator should be read “gets” and the equality operator should be read “double equals.”
- In C programs, **white-space characters** such as tabs, newlines and spaces are normally **ignored**. So, statements may be split over several lines. It's not correct to split identifiers.
- **Keywords** (p. 89; or reserved words) have special meaning to the C compiler, so you cannot use them as identifiers such as variable names.

Section 2.7 Secure C Programming

- One practice to help avoid leaving systems open to attacks is to avoid using `printf` with a single string argument.
- To display a string followed by a newline character, use the **puts function** (p. 90), which displays its string argument followed by a newline character.
- To display a string without a trailing newline character, you can use `printf` with the `"%s"` conversion specifier (p. 90) as the first argument and the string to display as the second argument.

Self-Review Exercises

2.1 Fill in the blanks in each of the following.

- Every C program begins execution at the function _____.
- Every function's body begins with _____ and ends with _____.
- Every statement ends with a(n) _____.
- The _____ standard library function displays information on the screen.
- The escape sequence `\n` represents the _____ character, which causes the cursor to position to the beginning of the next line on the screen.
- The _____ Standard Library function is used to obtain data from the keyboard.
- The conversion specifier _____ is used in a `scanf` format control string to indicate that an integer will be input and in a `printf` format control string to indicate that an integer will be output.
- Whenever a new value is placed in a memory location, that value overrides the previous value in that location. This process is said to be _____.
- When a value is read from a memory location, the value in that location is preserved; this process is said to be _____.
- The _____ statement is used to make decisions.

2.2 State whether each of the following is *true* or *false*. If *false*, explain why.

- Function `printf` always begins printing at the beginning of a new line.
- Comments cause the computer to display the text after `//` on the screen when the program is executed.
- The escape sequence `\n` when used in a `printf` format control string causes the cursor to position to the beginning of the next line on the screen.
- All variables must be defined before they're used.
- All variables must be given a type when they're defined.
- C considers the variables `number` and `NUMBER` to be identical.
- Definitions can appear anywhere in the body of a function.
- All arguments following the format control string in a `printf` function must be preceded by an ampersand (`&`).
- The remainder operator (`%`) can be used only with integer operands.
- The arithmetic operators `*`, `/`, `%`, `+` and `-` all have the same level of precedence.
- A program that prints three lines of output must contain three `printf` statements.

2.3 Write a single C statement to accomplish each of the following:

- Define the variables `c`, `thisVariable`, `q76354` and `number` to be of type `int`.
- Prompt the user to enter an integer. End your prompting message with a colon (`:`) followed by a space and leave the cursor positioned after the space.
- Read an integer from the keyboard and store the value entered in integer variable `a`.
- If `number` is not equal to 7, print "The variable `number` is not equal to 7."
- Print the message "This is a C program." on one line.
- Print the message "This is a C program." on two lines so that the first line ends with C.

- g) Print the message "This is a C program." with each word on a separate line.
 - h) Print the message "This is a C program." with the words separated by tabs.
- 2.4** Write a statement (or comment) to accomplish each of the following:
- a) State that a program will calculate the product of three integers.
 - b) Prompt the user to enter three integers.
 - c) Define the variables x, y and z to be of type int.
 - d) Read three integers from the keyboard and store them in the variables x, y and z.
 - e) Define the variable `result`, compute the product of the integers in the variables x, y and z, and use that product to initialize the variable `result`.
 - f) Print "The product is" followed by the value of the integer variable `result`.
- 2.5** Using the statements you wrote in Exercise 2.4, write a complete program that calculates the product of three integers.
- 2.6** Identify and correct the errors in each of the following statements:
- a) `printf( "The value is %d\n", &number );`
 - b) `scanf( "%d%d", &number1, number2 );`
 - c) `if ( c < 7 );{`
`printf( "C is less than 7\n" );`
`}`
 - d) `if ( c == 7 ) {`
`printf( "C is greater than or equal to 7\n" );`
`}`

Answers to Self-Review Exercises

- 2.1** a) main. b) left brace ({), right brace (}). c) semicolon. d) printf. e) newline. f) scanf. g) %d. h) destructive. i) nondestructive. j) if.
- 2.2**
- a) False. Function `printf` always begins printing where the cursor is positioned, and this may be anywhere on a line of the screen.
 - b) False. Comments do not cause any action to be performed when the program is executed. They're used to document programs and improve their readability.
 - c) True.
 - d) True.
 - e) True.
 - f) False. C is case sensitive, so these variables are unique.
 - g) True.
 - h) False. Arguments in a `printf` function ordinarily should not be preceded by an ampersand. Arguments following the format control string in a `scanf` function ordinarily should be preceded by an ampersand. We'll discuss exceptions to these rules in Chapter 6 and Chapter 7.
 - i) True.
 - j) False. The operators `*`, `/` and `%` are on the same level of precedence, and the operators `+` and `-` are on a lower level of precedence.
 - k) False. A `printf` statement with multiple `\n` escape sequences can print several lines.
- 2.3**
- a) `int c, thisVariable, q76354, number;`
 - b) `printf( "Enter an integer: " );`
 - c) `scanf( "%d", &a );`
 - d) `if ( number != 7 ) {`
`printf( "The variable number is not equal to 7.\n" );`
`}`

- e) `printf( "This is a C program.\n" );`
- f) `printf( "This is a C\nprogram.\n" );`
- g) `printf( "This\nis\nna\nC\nprogram.\n" );`
- h) `printf( "This\tis\ta\tC\tprogram.\n" );`

- 2.4**
- a) `// Calculate the product of three integers`
 - b) `printf( "Enter three integers: " );`
 - c) `int x, y, z;`
 - d) `scanf( "%d%d%d", &x, &y, &z );`
 - e) `int result = x * y * z;`
 - f) `printf( "The product is %d\n", result );`

- 2.5** See below.

```

1 // Calculate the product of three integers
2 #include <stdio.h>
3
4 int main( void )
5 {
6     printf( "Enter three integers: " ); // prompt
7
8     int x, y, z; // declare variables
9     scanf( "%d%d%d", &x, &y, &z ); // read three integers
10
11     int result = x * y * z; // multiply values
12     printf( "The product is %d\n", result ); // display result
13 } // end function main

```

- 2.6**
- a) Error: `&number`.
Correction: Eliminate the `&`. We discuss exceptions to this later.
 - b) Error: `number2` does not have an ampersand.
Correction: `number2` should be `&number2`. Later in the text we discuss exceptions to this.
 - c) Error: Semicolon after the right parenthesis of the condition in the `if` statement.
Correction: Remove the semicolon after the right parenthesis. [Note: The result of this error is that the `printf` statement will be executed whether or not the condition in the `if` statement is true. The semicolon after the right parenthesis is considered an empty statement—a statement that does nothing.]
 - d) Error: `=>` is not an operator in C.
Correction: The relational operator `=>` should be changed to `>=` (greater than or equal to).

Exercises

- 2.7** Identify and correct the errors in each of the following statements. (Note: There may be more than one error per statement.)

- a) `scanf( "&d", %value );`
- b) `printf( "The sum of %c and %c is %c /n", x, y );`
- c) `a + b + c = sum;`
- d) `if ( number >= largest );`
 `largest == number;`
- e) `\\ Program to determine the largest of three integers`
- f) `scanf( "%f", float );`
- g) `printf( "Remainder of %d divided by %d is \n", x, y, x / y );`
- h) `if ( x => y );`
 `printf( "%d is greater than or equal to %d\n, x, y" );`

- i) `print( "The product is &d\n," x * y );`
- j) `scanf( "%d, %d, %d", &x &y &z );`

2.8 Fill in the blanks in each of the following:

- a) All _____ must be declared before being used in a program.
- b) C is _____. Uppercase and lowercase letters are different in C.
- c) Single-line comments begin with _____.
- d) _____ are words reserved by C and cannot be used.
- e) _____ and _____ are ignored by the compiler.

2.9 Write a single C statement or line that accomplishes each of the following:

- a) Print the message "Have a nice day."
- b) Assign the sum of variables b and c to variable a.
- c) Check if the value of variable a is greater than variable b. If it is, store the difference of the two in variable c.
- d) Input three integer values from the keyboard and place them in int variables p, q, and r.

2.10 State which of the following are *true* and which are *false*. If *false*, explain your answer.

- a) C regards the functions `main` and `Main` as identical.
- b) The associativity of the operators specifies whether they evaluate from left to right or from right to left.
- c) The statement `if(a = b)` checks whether the variables a and b are equal.
- d) Conditions in `if` statements are formed by using assignment operators.
- e) The following are all valid variable names: `_3g`, `my_val`, `h22`, `123greetings`, `July98`.

2.11 Fill in the blanks in each of the following:

- a) The _____ statement allows a program to perform different actions based on a condition.
- b) If the result of an integer division, where both the numerator and the denominator are integers, is a fraction, the fractional part is _____.
- c) The directive _____ tells the preprocessor to include the contents of the input/output stream header files.

2.12 What, if anything, prints when each of the following statements is performed? If nothing prints, then answer "Nothing." Assume `a = 15`, `b = 4` and `c = 7`.

- a) `printf( "%d", a % b );`
- b) `printf( "%d", a % c + b );`
- c) `printf( "b=" );`
- d) `printf( "a = 15" );`
- e) `printf( "%d = a + b", a + b );`
- f) `c = a + b;`
- g) `scanf( "%d%d", &a, &b );`
- h) `// printf( "Now a and b changes to %d and %d", a, b );`
- i) `printf( "\n" );`

2.13 Which, if any, of the following C statements contain variables whose values are replaced?

- a) `printf( "Enter two Numbers: " );`
- b) `scanf( "%d%d ", &a, &b );`
- c) `sum = a + b;`
- d) `printf( "\nThe result is: %d ", sum );`

2.14 Given the equation $y = ax^3 - bx^2 - 6$, which of the following, if any, are correct C statements for this equation?

- a) `y = a * x * x * x - b * x * x - 6;`
- b) `y = a * x * x * x * b * x * x - 6;`
- c) `a * (x * x * x) - b * x * x * (- 6 );`

- d) $a * (x * x * x) - b * (x * x) - 6;$
 e) $a * x * x * x - (b * x * x - 6);$
 f) $(a * x * 3 - b * x * 2) - 6;$

2.15 State the order of evaluation of the operators in each of the following C statements and show the value of x after each statement is performed.

- a) $x = 8 + 15 * (6 - 2) - 1;$
 b) $x = 5 \% 5 + 5 * 5 - 5 / 5;$
 c) $x = (5 * 7 * (5 + (7 * 5 / (7))));$

2.16 (*Arithmetic*) Write a program that asks the user to enter two numbers, obtains them from the user and prints their sum, product, difference, quotient and remainder.

2.17 (*Final Velocity*) Write a program than asks the user to enter the initial velocity and acceleration of an object, and the time that has elapsed, places them in the variables u , a , and t , and prints the final velocity, v , and distance traversed, s , using the following equations.

- a) $v = u + at$
 b) $s = ut + \frac{1}{2}at^2$

2.18 (*Comparing Values*) Write a program that asks the user to enter the highest rainfall ever in one season for a country, and the rainfall in the current year for that country, obtains the values from the user, checks if the current rainfall exceed the highest rainfall and prints an appropriate message on the screen. If the current rainfall is higher, it assigns that value as the highest rainfall ever. Use only the single-selection form of the `if` statement you learned in this chapter.

2.19 (*Arithmetic, Largest Value and Smallest Value*) Write a program that inputs three different integers from the keyboard, then prints the sum, the average, the product, the smallest and the largest of these numbers. Use only the single-selection form of the `if` statement you learned in this chapter. The screen dialogue should appear as follows:

```
Enter three different integers: 13 27 14
Sum is 54
Average is 18
Product is 4914
Smallest is 13
Largest is 27
```

2.20 (*Converting from seconds to hours, minutes and seconds*) Write a program that asks the user to enter the total time elapsed, in seconds, since an event and converts the time to hours, minutes and seconds. The time should be displayed as hours:minutes:seconds. [*Hint*: Use the remainder operator]

2.21 (*Shapes with Asterisks*) Write a program that prints the following shapes with asterisks.

```
*****          ***          *          *
*              *      *      ****         * *
*              *      *      *****        *  *
*              *      *      *              *  *
*              *      *      *              *  *
*              *      *      *              *  *
*              *      *      *              *  *
*              *      *      *              *  *
*              *      *      *              *  *
*****          ***          *          *
```

2.22 What does the following code print?

```
printf( "%\n**\n***\n****\n*****\n" );
```

2.23 (*Largest and Smallest Integers*) Write a program that reads in three integers and then determines and prints the largest and the smallest integers in the group. Use only the programming techniques you have learned in this chapter.

2.24 (*Odd or Even*) Write a program that reads an integer and determines and prints whether it's odd or even. [*Hint*: Use the remainder operator. An even number is a multiple of two. Any multiple of two leaves a remainder of zero when divided by 2.]

2.25 Print your initials in block letters down the page. Construct each block letter out of the letter it represents, as shown on the top of the next page:

```

PPPPPPPP
 P   P
 P   P
 P   P
  P P

 JJ
 J
J
 J
  JJJJJJ

DDDDDDDD
D       D
D       D
D       D
  DDDD

```

2.26 (*Multiples*) Write a program that reads in two integers and determines and prints whether the first is a multiple of the second. [*Hint*: Use the remainder operator.]

2.27 (*Checkerboard Pattern of Asterisks*) Display the following checkerboard pattern with eight `printf` statements and then display the same pattern with as few `printf` statements as possible.

```

* * * * *
 * * * * *
* * * * *
 * * * * *
* * * * *
 * * * * *
* * * * *
 * * * * *

```

2.28 Distinguish between the terms fatal error and nonfatal error. Why might you prefer to experience a fatal error rather than a nonfatal error?

2.29 (*Integer Value of a Character*) Here's a peek ahead. In this chapter you learned about integers and the type `int`. C can also represent uppercase letters, lowercase letters and a considerable variety of special symbols. C uses small integers internally to represent each different character. The set of characters a computer uses together with the corresponding integer representations for those characters is called that computer's character set. You can print the integer equivalent of uppercase A, for example, by executing the statement

```
printf( "%d", 'A' );
```

Write a C program that prints the integer equivalents of some uppercase letters, lowercase letters, digits and special symbols. As a minimum, determine the integer equivalents of the following: A B C a b c 0 1 2 \$ * + / and the blank character.

2.30 (*Separating Digits in an Integer*) Write a program that inputs one five-digit number, separates the number into its individual digits and prints the digits separated from one another by three spaces each. [Hint: Use combinations of integer division and the remainder operation.] For example, if the user types in 42139, the program should print

```
4   2   1   3   9
```

2.31 (*Table of Squares and Cubes*) Using only the techniques you learned in this chapter, write a program that calculates the squares and cubes of the numbers from 0 to 10 and uses tabs to print the following table of values:

number	square	cube
0	0	0
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
6	36	216
7	49	343
8	64	512
9	81	729
10	100	1000

Making a Difference

2.32 (*Body Mass Index Calculator*) We introduced the body mass index (BMI) calculator in Exercise 1.12. The formulas for calculating BMI are

$$BMI = \frac{weightInPounds \times 703}{heightInInches \times heightInInches}$$

or

$$BMI = \frac{weightInKilograms}{heightInMeters \times heightInMeters}$$

Create a BMI calculator application that reads the user's weight in pounds and height in inches (or, if you prefer, the user's weight in kilograms and height in meters), then calculates and displays the user's body mass index. Also, the application should display the following information from the Department of Health and Human Services/National Institutes of Health so the user can evaluate his/her BMI:

```
BMI VALUES
Underweight: less than 18.5
Normal:      between 18.5 and 24.9
Overweight:  between 25 and 29.9
Obese:       30 or greater
```

[Note: In this chapter, you learned to use the `int` type to represent whole numbers. The BMI calculations when done with `int` values will both produce whole-number results. In Chapter 4 you'll learn to use the `double` type to represent numbers with decimal points. When the BMI calculations are performed with `doubles`, they'll both produce numbers with decimal points—these are called “floating-point” numbers.]

2.33 (*Car-Pool Savings Calculator*) Research several car-pooling websites. Create an application that calculates your daily driving cost, so that you can estimate how much money could be saved by car pooling, which also has other advantages such as reducing carbon emissions and reducing traffic congestion. The application should input the following information and display the user's cost per day of driving to work:

- a) Total miles driven per day.
- b) Cost per gallon of gasoline.
- c) Average miles per gallon.
- d) Parking fees per day.
- e) Tolls per day.